WinPE Build & USB Runbook (post-WDS migration)

Server: pc-deploy (192.168.5.141) · **Last verified:** 2026-08-07 **Scope:** How the WinPE deployment boot image is built, what goes in it, how it reaches machines (PXE + USB), and the exact steps to (re)build a bootable USB.

> Read this before touching the imaging boot image. The pre-July-2026 tooling (`01c-build-winpe.ps1` , tftpd64, `C:\WinPE_amd64` , `WimUpdate4`) is **dead** — see "Retired / do-not-use" at the bottom. Everything live now lives in `C:\WinPE-src\` on pc-deploy.

1. Architecture at a glance

There is **one** boot image, delivered **two** ways:

```
          C:\WinPE-src\  (the build workshop, all SYSTEM scheduled tasks)
               |
build.ps1  -> winpe-deploy-final.wim   (OCs, deploy scripts, startnet, OpenSSH, WPF screen, fonts)
(+ driver/NIC inject scripts add storage/NIC .inf on top)
               |
               +-------------------------------+
               |                               |
     PXE (network boot)                 USB (physical stick)
     served by WDS from:               written by usb.ps1 from:
C:\RemoteInstall\Boot\x64\Images\        C:\WinPE-src\winpe-deploy-final.wim
     winpe-deploy-final.wim              -> Disk 1 (SanDisk), label JUNIPER-PE
```

- **PXE is served by WDS**, not TFTP. The old **tftpd64 stack is retired** (`C:\tftpd64` no longer exists).
- The two copies of `winpe-deploy-final.wim` (WDS at `C:\RemoteInstall\...` and the workshop at `C:\WinPE-src\...`) must be kept **identical**. They drifted before (see §7); always deploy the same file to both and hash-check.

Boot flow on the target machine

`startnet.cmd` → `wpeinit` → starts `sshd` (best-effort) → runs `X:\Windows\System32\deploy-boot.ps1` . `deploy-boot.ps1` waits for network, maps `\\pc-deploy\deploy$` as `junadmin` , then shows the **branded WPF imaging screen** (`X:\deploy-screen.ps1`) while `deploy.ps1` runs **hidden** and appends `X:\deploy_log.txt` , which the screen tails to advance its 14 phase ticks. If WPF can't

load, `deploy-boot.ps1` falls back to the **console** `deploy.ps1`. The full deploy logic (`deploy.ps1`) lives on the share, so day-to-day changes to it do **not** require a WIM rebuild — only changes to `deploy-boot.ps1`, `startnet.cmd`, `toolkit.ps1`, `deploy-screen.ps1`, the fonts, or injected drivers require a rebuild.

Why WPF, not a browser

The imaging screen is a native **PowerShell + WPF/XAML** window — the standard approach for WinPE deployment UIs (SCCM/MDT/OSD shops; see OSDProgress / OSDCloud-Progress). **Full Edge/Chromium does not run in stock WinPE** (it needs OS components WinPE lacks — it launches and silently exits). WPF needs only `WinPE-NetFx` + `WinPE-PowerShell`, both already in the image, so it renders reliably and keeps the WIM small (~792 MB vs ~1.4 GB with Edge bundled). HTAs (Trident/IE11) are the older path and can't render the modern design's CSS.

---

2. The build workshop — `C:\WinPE-src\`

File	Runs as	Purpose
`build.ps1` (`WINPE-Build` task)	SYSTEM	Master build from `winre-base.wim`: add OCs (NetFx/PowerShell/DismCmdlets), drop `winpeshl.ini`, bake `deploy-boot.ps1` + `toolkit.ps1` + `deploy-screen.ps1` + `brand-fonts\` (from `C:\deploy\scripts\winpe\`), write `startnet.cmd`, bake OpenSSH + host keys, export compressed `winpe-deploy-final.wim`.
`bake.ps1`	SYSTEM	Overlay the credentialed `deploy-boot-cred.ps1` (junadmin password) into the final WIM.
`usb.ps1`	admin	**The USB writer.** ADK media tree in `C:\WinPE_usb\media`, copies the WIM as `boot.wim`, CA-2023 Secure Boot loader, fixes media BCD, auto-detects the single USB disk (guards: USB bus, <64 GB, not system/boot), formats FAT32 `JUNIPER-PE`, robocopies.
NIC/storage inject scripts (`nic-only.ps1`, `intel-nic.ps1`, `realtek-nic.ps1`, `winpe-nic-inject.ps1`)	SYSTEM	Mount the WIM and `Add-WindowsDriver` storage/NIC `.inf`. **Run AFTER `build.ps1`** — a fresh `build.ps1` does NOT include them.
`inspect*.ps1`, `wim-verify.ps1`	SYSTEM	Read-only WIM sanity checks.
`deploy-screen.ps1` (baked to `X:\deploy-screen.ps1`)	—	The WPF branded imaging screen. Tails `X:\deploy_log.txt`; loads `X:\brand-fonts\` as private fonts; renders the wordmark from `wordmark.pathdata`.
`brand-fonts\` (baked to `X:\brand-fonts\`)	—	`GoogleSans-Variable.ttf`, `SourceSerif4-Variable.ttf`, `RobotoMono-Variable.ttf`, `wordmark.pathdata`.

Key consequence: because drivers are injected *after* the base build, **never** promote a raw `build.ps1` output without re-injecting the current driver set — or you regress "can't see the disk / no network" on machines whose NICs/storage need those `.inf`s.

---

3. Drivers in the live image (baseline 2026-08-07)

7 third-party `[Net]` drivers (verify with `Get-WindowsDriver -Path <mount>`):

- Intel `e1r68x64.inf` (12.18.11.1 and 12.18.9.6)
- Intel `e1dn.inf` (20.0.2.19)
- Realtek `ws640x64.inf` (10.50.511.2021)
- Microsoft USB-C RNDIS dongle: `netrndis.inf`, `usbnet.inf`, `rndiscmp.inf`

If a rebuild shows fewer than 7, the NIC-inject step was skipped — re-run it before promoting.

---

4. Credential handling (junadmin password)

`deploy-boot.ps1` carries `$DeployPass = '##WINPE_PASS##'` as a **placeholder**; the real junadmin password is substituted only into `deploy-boot-cred.ps1` and baked into the WIM. Rules:

- The placeholder must **never** be replaced with a real password in any file in git or on the share. Only the in-WIM copy holds the real value.
- Substitute with a literal string replace (`.Replace()`), never regex `-replace` (a `$` in the password would corrupt it).
- Source of the password: the junadmin credential (`push-cred.xml` PSCredential, or `op://Private/pc-deploy/password`). Never echo it. Scrub `deploy-boot-cred.ps1` after baking.

---

5. RUNBOOK — build a bootable USB stick

Prereqs: run on pc-deploy (or remote as junadmin/SYSTEM), USB stick inserted (the only USB disk present), ADK + WinPE add-on installed, live WIM present.

5a. If you only changed `deploy.ps1` (share logic)

Nothing to rebuild. `deploy.ps1` is read from `\\pc-deploy\deploy$\scripts\` at boot. Just re-run the deploy.

5b. If you changed `deploy-boot.ps1`, `startnet.cmd`, `toolkit.ps1`, `deploy-screen.ps1`, the fonts, or drivers

You must update the WIM, then write media.

1. **Update the source files** in `C:\deploy\scripts\winpe\` (`deploy-boot.ps1` with the `##WINPE_PASS##` placeholder, `toolkit.ps1`, `deploy-screen.ps1`, `brand-fonts\`).
2. **Produce the credentialed boot script:** write `C:\WinPE-src\deploy-boot-cred.ps1` = the new `deploy-boot.ps1` with the junadmin password substituted (see §4).
3. **Patch the live image (driver-safe method):** copy the current live WIM to a work copy, mount it, then:
4. overwrite `\Windows\System32\deploy-boot.ps1` with the credentialed version,
5. copy `deploy-screen.ps1` to the WIM root (`X:\deploy-screen.ps1`),
6. copy the fonts to `X:\brand-fonts\` (3 TTFs + `wordmark.pathdata`),
7. (SSH) ensure `\Windows\System32\OpenSSH\*`, host keys in `\ProgramData\ssh`, and the `sshd` block in `startnet.cmd` are present,
8. `Dismount-WindowsImage -Save`, then `Export-WindowsImage -CompressionType max` to a final file, and **verify** (mount read-only: `deploy-boot.ps1` has no `##WINPE_PASS##` and parses clean, `deploy-screen.ps1` present + parses, `brand-fonts\GoogleSans-Variable.ttf` + `wordmark.pathdata` present, `edge` folder absent, `sshd.exe` present, `Get-WindowsDriver` count == expected).

   > `C:\WinPE-src\wpf-bake.ps1` is the exact reproducible script for this. A from-scratch `build.ps1` is the alternative, but only if you re-inject the full driver set afterward (§2–§3).

9. **Promote the verified WIM to BOTH targets, hash-verified identical:**
10. `C:\WinPE-src\winpe-deploy-final.wim` (USB source)
11. `C:\RemoteInstall\Boot\x64\Images\winpe-deploy-final.wim` (PXE) — stop `WDSServer`, copy, start `WDSServer`. Keep a `.bak-*` of the prior WIM.
12. **Write the stick:** run `C:\WinPE-src\usb.ps1` (auto-detects the single USB disk, formats `JUNIPER-PE`, writes media).
13. **HASH-VERIFY THE STICK (mandatory):** flush the volume cache (`Write-VolumeCache`), then compare SHA256 of `<USB>:\sources\boot.wim` against the source WIM. They **must** match. (`C:\WinPE-src\usb-verify.ps1` does this.)
14. **(Optional) Refresh the legacy media tree** so the old path can't hand out a stale stick: run `01f-refresh-usb-media.ps1` (republishes `C:\deploy\winpe-media`).

5c. Booting a target machine

UEFI boot from the stick (or PXE). **Secure Boot may be left ON** — the media uses the CA-2023 signed `bootmgfw_EX`. At the WinPE menu: `[T]` = toolkit, `[D]` = deploy now, otherwise auto-deploys after 60 s. The WPF screen shows the branded progress; `Esc` closes it; if WPF can't load it falls back to the console.

---

6. Image size

The WPF screen is native and tiny; the image is **~792 MB**. (An earlier experiment bundled Microsoft Edge to render an HTML screen — that pushed the WIM to ~1.4 GB and, worse, Edge does not run in WinPE. Both were removed. Do not re-add a browser; see §1 "Why WPF".)

---

7. Findings / gotchas (so we stop re-chasing these)

- **WDS migration (2026-07-21):** moved off tftpd64 to WDS. Live boot image is now `C:\RemoteInstall\Boot\x64\Images\winpe-deploy-final.wim`.
- **PXE = WDS, not TFTP.** Don't look for tftpd64; it's gone.
- **Two WIM copies drift.** PXE (`RemoteInstall`) and the workshop (`WinPE-src`) had diverged both directions — the from-scratch `build.ps1` had dropped 3 NIC drivers (both Intel `e1r68x64` + Realtek `ws640x64`) because it rebuilds from pristine `winre-base.wim` and the NIC-inject scripts run afterward. **Always deploy the same verified file to both paths and hash-check.**
- `build.ps1` **injects no drivers.** Patch the known-good live WIM (driver-safe) or re-inject drivers after a from-scratch build.
- **Edge/Chromium does NOT run in WinPE** — it launches and silently exits (missing OS components). The branded screen is native **PowerShell/WPF** instead (`deploy-screen.ps1`), which needs only NetFx + PowerShell (already present). This is how SCCM/MDT/OSD shops do it. Refs: windows-noob "WPF in WinPE", ChrisStro/OSDProgress, agovardhanam/OSDCloud-Progress.
- **WPF private fonts:** the design fonts are loaded from a folder without installing — `FontFamily = "file:///X:/brand-fonts/#Google Sans"` (family names: `Google Sans`, `Source Serif 4`, `Roboto Mono`). The wordmark is drawn from vector path data (`wordmark.pathdata`, `Geometry.Parse`).
- `##WINPE_PASS##` **discipline** — see §4. Never commit a real password.
- **Always hash-verify a freshly written stick** — see §5 step 6.
- **DISM as SYSTEM:** WIM mount/dismount runs as SYSTEM (scheduled task) or a normal elevated local process (CIM `Win32_Process.Create` as junadmin). WinRM's process isolation breaks DISM dismount — don't run the DISM steps over WinRM.

Retired / do-not-use (pre-migration tooling — slated for cleanup)

- `01c-build-winpe.ps1` (built into `C:\WinPE_amd64` + tftpd64) — dead path.
- tftpd64 stack / `C:\tftpd64` — removed.
- `WimUpdate4` task + `wim-update4.ps1` / `wim-update-deploy-boot.ps1` — superseded by `build.ps1` / `bake.ps1`.
- `winpemedia$` / `tftpd64$` shares — gone.
- Edge-kiosk imaging screen + staged Edge (`imaging-screen.html`, `X:\edge`) — replaced by the WPF screen; Edge does not run in WinPE.

Deploy-time input flow + pre-wipe backup (2026-08-07)

"Console prompts first, then branded screen" (Option 3): `deploy-boot.ps1` runs `deploy.ps1` in the FOREGROUND (console visible); all operator decisions happen in the console, then `deploy.ps1` launches the WPF screen (`X:\deploy-screen.ps1`) right before partitioning and keeps logging to `X:\deploy_log.txt` (which the screen tails).

- KNOWN host (serial/MAC resolves in inventory): computer name + Windows edition auto-accept via 10s countdowns; the pre-wipe backup gate also counts down. Hands-off.

- UNKNOWN machine: a "MACHINE NOT IN INVENTORY - UNKNOWN" banner shows and the name/edition fields WAIT for input (no auto-accept).

Pre-wipe user-data backup (two-part; see design-prewipe-backup.md): - Armed per-device from the inventory UI ("Back up at next re-imaging"), OR opted-in at the console gate (B = back up first when not armed; S = skip when armed; 10s). - Captures each profile's Desktop/Documents/Downloads/Pictures/Videos/Favorites + Edge/Chrome bookmarks + Outlook (minus .ost) + C:\dev to \pc-deploy\backup$\\winpe-, over the existing junadmin session. - FAIL-SAFE: if a device is ARMED and the backup FAILS, imaging HALTS before wipe. - Wiring: Write-DeployLog also appends X:\deploy_log.txt (the screen source); the backup hook sits immediately before diskpart clean. Server endpoints: /ingest/backup/{for-device,begin,start,finish} (RFC-1918).

Regression note: the backup hook was stripped from deploy.ps1 by the Aug 5-7 stagebios/hardening edits and RESTORED 2026-08-07 from deploy.ps1.bak-stagebios. deploy.ps1 lives on the deploy share (read at runtime) - changes take effect without a WIM rebuild; only deploy-boot.ps1 / deploy-screen.ps1 / fonts require a rebake.